

RESEARCH-FLOW-TEMPLATE

Claude Code Wiring Guide

How hooks, skills, CLAUDE.md, and folder structure combine to make an agentic research system.

A simple starter — update the flow, skills, and hooks to fit your project.

CLAUDE.md

hooks

skills

memory/

sources/

— OVERVIEW

What's in this deck

CONTEXT & WHY

01	Why This System Exists	motivation
02	The Common Research Flow	lifecycle
03	The Mental Model	CLAUDE.md · hooks · skills
04	CLAUDE.md Template	what it looks like

CLAUDE CODE PRIMITIVES

05	What is Claude Code?	foundations
06	CLAUDE.md — The Brain	always-loaded context

CLAUDE CODE PRIMITIVES (CONT.)

07	.claude/ Folder Layout	config directory
08-10	settings.json + Hooks (3 slides)	wiring & guardrails
11	Skills — Slash Commands	custom commands
12	Repo Folder Layout	project structure
13	A Full Research Session	end-to-end
14-15	Adding Your Own Hooks & Skills	extending
16-17	/import — Existing Project Onboarding	new skill

GET STARTED

18	Prerequisites	tools & env
19	Quick Start	from zero to research

— MOTIVATION

Why This **System** Exists

THE PROBLEM

- ✗ Research creates **hundreds of artifacts** — papers, notes, experiments, results — with no thread connecting them to the original question.
- ✗ LLMs are **stateless**. Great at reasoning in the moment; terrible at remembering what they concluded last week. Every session starts from scratch.
- ✗ Without structure, Claude will **hallucinate continuity** — appearing to remember context it has actually lost, or repeating work already done.

THE SOLUTION

- ✓ **Persistent memory in files.** Every insight is written down. Claude reads it fresh each session — no hallucinated continuity.
- ✓ **A defined methodology.** CLAUDE.md tells Claude exactly how to do research in this project — what structure to follow, what quality bar to meet.
- ✓ **Automated guardrails.** Hooks enforce the rules Claude might otherwise skip under pressure. The structure holds even in long sessions.

Core insight: Claude is not the researcher. Claude is the *research assistant*. The methodology lives in the repo — Claude executes it. This means the research process is **reproducible, auditable, and improvable** independently of which Claude session is running.

WITHOUT THIS SYSTEM

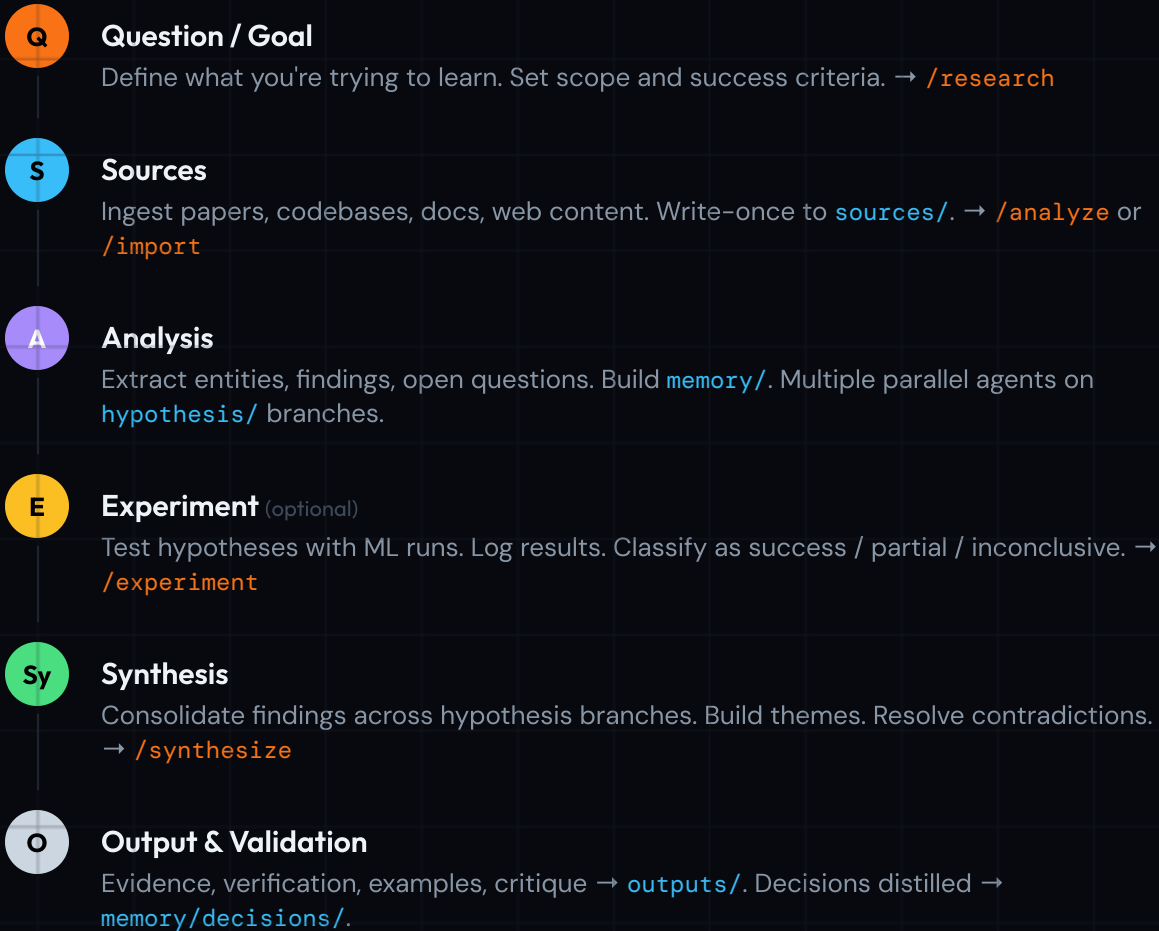
Session 1: "Let's research X" → findings in chat
 Session 2: Claude has no memory → starts over
 Session 3: Contradicts Session 1's conclusions
 Week 3: No idea what was already tried
 → **Research is lost between sessions**

WITH THIS SYSTEM

Every session: Claude reads memory/ and CLAUDE.md
 Every insight: written to memory/findings/
 Every contradiction: surfaced in open-questions/
 Any session: full context immediately available
 → **Research compounds across sessions**

LIFECYCLE

The Common Research Flow



EACH PHASE MAPS TO A SKILL

PHASE	SKILL
Goal definition	<code>/research</code>
Source ingest	<code>/analyze</code> , <code>/import</code> , <code>/read-pdf</code>
Experiment	<code>/experiment</code>
Consolidation	<code>/synthesize</code> , <code>/distill</code>
Validation	<code>/evidence</code> , <code>/verify</code> , <code>/examples</code> , <code>/critique</code>
Maintenance	<code>/lint</code>

Parallel agents: Multiple `hypothesis/` branches can explore different angles simultaneously. Each agent reads the same `CLAUDE.md`, writes to its own branch, and a synthesis agent merges the best findings.

— MENTAL MODEL

Three Primitives, **One System**

01

CLAUDE.md THE METHODOLOGY

Guidance & flow for research

Defines *what good research looks like* in this project — what goes in memory, how findings are structured, what quality bar to meet, which git branches to use. Claude reads this at every session start. It's the research methodology, not code.

02

Hooks THE GUARDRAILS

Force Claude to strictly follow the methodology

Shell scripts that intercept Claude's tool calls and either block or remind. Claude can't "forget" a rule — the hook runs automatically regardless of context length or session state. Hooks encode the invariants that *must never break*.

03

Skills THE PROCEDURES

Detailed guidance for each operation

Step-by-step instructions for each research phase — loaded only when invoked. Skills don't define policy (that's CLAUDE.md); they define *how to execute* a phase: what to ask, what to write, what to log, how to commit.

— WHAT IT LOOKS LIKE

A CLAUDE.md Template

```
# My Research Project
```

CLAUDE.MD

```
## Research Goal
```

```
What question are we answering?
```

```
What counts as done (success criteria)?
```

```
## Memory Structure
```

```
sources/      - raw material, write-once, never edited
```

```
memory/       - wiki: entities, findings, themes, questions
```

```
outputs/      - deliverables: reports, evidence, decisions
```

```
## Memory Rules
```

- Every page: YAML frontmatter with `title`, `created`, `updated`, `confidence` (high/medium/low), `tags`
- `memory/index.md` updated after every ingest
- `memory/log.md` is append-only – never edit past entries
- Check `entity-registry.json` before creating entity pages

```
## Quality Standards
```

- Every finding needs a citation + counter-argument
- Classify each finding:
 - `source` - directly cited from material
 - `analysis` - inferred; show your reasoning
 - `unverified` - noted but not validated
 - `gap` - explicitly missing information

```
## Git Flow
```

- Never commit to `main`
- All work on typed branches:
 - `research/` - new goal

This file **IS** the research methodology. Claude reads it at startup every session. It doesn't need to be reminded of the rules — they're here.

WHAT BELONGS IN CLAUDE.MD

Rules that apply to **every operation** — memory hygiene, quality gates, git conventions, what counts as a finding vs. an entity.

WHAT DOES NOT BELONG

Step-by-step procedures for specific commands. Those go in **skills** — loaded only when invoked, keeping CLAUDE.md focused.

THE RELATIONSHIP

Skills reference CLAUDE.md implicitly. When `/analyze` says "write a finding", CLAUDE.md defines what a finding must contain.

Start minimal. Add sections only when Claude consistently gets something wrong. Every rule in CLAUDE.md should exist because something broke without it.

— FOUNDATIONS

What is Claude Code?

- 01 CLI agent, not a chat window. Claude Code runs in your terminal and has direct access to your filesystem and shell.
- 02 Reads and writes files directly. No copy-paste. It opens, edits, and creates files in your working directory.
- 03 Persists context across sessions via files — not conversation memory. `CLAUDE.md` and `memory/` survive between runs.
- 04 Extensible. Give it custom commands (skills) and automated guardrails (hooks) — both live in `.claude/`.

When you run `claude` in this repo, it loads `CLAUDE.md` and `.claude/settings.json` before anything else.

SHELL

```
# Start a session
claude

# Claude now has access to:
• your terminal
• all files in the current directory
• git, gh, uv, python, ...

# Its configuration lives in:
.claude/
CLAUDE.md

# Persistent research memory lives in:
memory/
sources/
goals/
```

— PERSISTENCE LAYER 0

CLAUDE.md — The Brain

- Loaded automatically at startup for every session in this directory. Think of it as a persistent system prompt.
- Defines the research loop — what each command is supposed to do, memory structure rules, Git conventions.
- Always in context. Skills are only loaded when invoked. If CLAUDE.md and a skill disagree, CLAUDE.md wins.

Rule: Everything Claude must *always* know goes in CLAUDE.md. Step-by-step procedures for specific commands go in skills.

CLAUDE.MD CONTAINS

Research Loop

/research → /analyze → /synthesize

The lifecycle of a research goal from question to deliverable.

Memory Rules

Frontmatter, index hygiene, log format

What every memory page must contain, and how to keep it consistent.

Git + Quality

Branch types, FUNGI counter-arguments

Branch naming, commit prefixes, confidence levels, verification gates.

— CONFIG DIRECTORY

The .claude/ Folder

```
.claude/
├── settings.json      # hook wiring — the router
├── hooks/
│   ├── block-source-modification.sh
│   │   PreToolUse → Edit — sources/ are immutable
│   ├── block-commit-protected-branch.sh
│   │   PreToolUse → Bash — no commits to main
│   └── post-memory-update.sh
│       PostToolUse → Edit|Write — sync index + log
└── skills/
    ├── research.md    # /research
    ├── analyze.md     # /analyze
    ├── synthesize.md  # /synthesize
    ├── experiment.md  # /experiment
    ├── distill.md     # /distill
    ├── evidence.md    # /evidence
    ├── verify.md      # /verify
    ├── examples.md    # /examples
    ├── critique.md    # /critique
    ├── lint.md        # /lint
    └── read-pdf/      # /read-pdf (+ helper scripts)
```

TREE

SETTINGS.JSON

The routing table. Wires tool events to shell scripts. Contains **no logic** — only configuration.

HOOKS/

Shell scripts that enforce invariants. Exit **0** = allow. Exit **1** = block with error.

SKILLS/

Markdown files that define **slash commands**. Loaded on-demand when invoked — not at startup.

settings.json — The Router

JSON

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit",
        "hooks": [{
          "type": "command",
          "command": "bash .claude/hooks/block-source-modification.sh Edit \"$TOOL_INPUT_FILE_PATH\""
        }]
      },
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": "bash .claude/hooks/block-commit-protected-branch.sh Bash \"$TOOL_INPUT_COMMAND\""
        }]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "bash .claude/hooks/post-memory-update.sh \"$TOOL_NAME\" \"$TOOL_INPUT_FILE_PATH\""
        }]
      }
    ]
  }
}
```

HOOK EVENTS	
EVENT	FIRES WHEN
PreToolUse	Before Claude calls an tool
PostToolUse	After a tool completes
ENV VARIABLES PASSED	
VARIABLE	VALUE
\$TOOL_NAME	Edit , Bash , Write ...
\$TOOL_INPUT_FILE_PATH	File path (Edit/Write)
\$TOOL_INPUT_COMMAND	Shell command (Bash)

Matcher syntax: Use | for OR — "Edit|Write" fires on either tool.

— HOOK MECHANICS

How Hooks **Control** Claude

2

exit 2 — Block

The tool call is **cancelled**. Claude receives stderr as the error message. *Only exit 2 blocks* — exit 1 is a non-blocking error and execution continues.

0

exit 0 — Allow

The tool proceeds. stdout is added to Claude's context — use for **reminders** (PostToolUse). JSON on stdout can carry structured decisions.

1

exit 1 — Non-blocking error

Execution **continues**. A notice appears in the transcript but the tool is not cancelled. Not useful for enforcement.

```
#!/bin/bash                                BASH - BLOCK PATTERN (EXIT 2 + STDERR)
if [[ "$FILE_PATH" == *"sources/"* ]]; then
    echo "BLOCKED: sources/ are immutable." >&2
    exit 2    # ← cancels the Edit
fi
exit 0       # ← allow
```

```
#!/bin/bash                                BASH - REMINDER PATTERN (EXIT 0 + STDOUT)
if [[ "$FILE_PATH" == *"memory/"* ]]; then
    # stdout at exit 0 → Claude's context
    echo "REMINDER: Update index.md + log.md"
fi
exit 0     # ← write still happens
```

Source: code.claude.com/docs/en/hooks — "Only exit 2 blocks actions. Exit 1 is treated as non-blocking error and execution continues."

— HOOK 1 OF 3

block-source-modification.sh

PreToolUse

Edit → sources/

WHAT IT DOES

Rejects any **Edit** call targeting a file under `sources/`.

WHY IT EXISTS

Sources are the raw material of research. Once ingested, they must never change — otherwise findings that cite them become untrustworthy.

If Claude wants to "fix" a source, it should create a finding in `memory/findings/` instead.

WHY EDIT BUT NOT WRITE

Write creates new files. **Edit** modifies existing ones. New source files are allowed; modifying them after creation is not.

BASH

```
#!/bin/bash
TOOL_NAME="$1"
FILE_PATH="$2"

# Only intercept Edit (not Write)
if [[ "$TOOL_NAME" != "Edit" ]]; then
    exit 0
fi

# Block edits to sources/ (skip .gitkeep)
if [[ "$FILE_PATH" == *"sources/"* \
    && "$FILE_PATH" != *.gitkeep* ]]; then
    echo "BLOCKED: Source files are immutable
after creation. Create a finding in
memory/findings/ instead."
    exit 1
fi
```

— HOOK 2 OF 3

block-commit-protected-branch.sh

PreToolUse

Bash → git commit

WHAT IT DOES

Rejects **git commit** commands when the current branch is `main`.

WHY IT EXISTS

All research work happens on typed branches — `research/`, `hypothesis/`, `synthesis/`, `review/`.

Committing to main directly bypasses issue-tracking and the review workflow. The hook enforces this automatically rather than relying on Claude remembering the rule.

Branch naming: `hypothesis/GH-7-attention-mechanisms`,
`synthesis/GH-5-consolidate`

BASH

```
#!/bin/bash
COMMAND="$2"

# Only intercept Bash calls
if [[ "$1" != "Bash" ]]; then
    exit 0
fi

# Check for git commit in the command
if [[ "$COMMAND" == *"git commit"* ]]; then
    BRANCH=$(git rev-parse --abbrev-ref HEAD 2>/dev/null)
    if [[ "$BRANCH" == "main" ]]; then
        echo "BLOCKED: Cannot commit to main.
Create a branch first:
    research/, hypothesis/,
    synthesis/, or review/"
        exit 1
    fi
fi
```

— HOOK 3 OF 3

post-memory-update.sh

PostToolUse

Edit|Write → memory/

WHAT IT DOES

After any file write inside `memory/`, prints a reminder to update `memory/index.md` and append to `memory/log.md`.

WHY IT EXISTS

The index and log are the **consistency layer** for working memory. Without them, pages become orphans (not findable) or operations go unrecorded.

This hook ensures Claude does both in the *same turn* — it can't skip them and move on.

WHY EXIT 0, NOT EXIT 1

This is a **reminder**, not a block. The write already happened. Claude sees the output and acts on it in the same response.

BASH

```
#!/bin/bash
TOOL_NAME="$1"
FILE_PATH="$2"

if [[ "$TOOL_NAME" != "Edit" \
    && "$TOOL_NAME" != "Write" ]]; then
    exit 0
fi

# Check for memory/ writes,
# excluding the index/log/registry
if [[
    "$FILE_PATH" == *"memory/"* \
    && "$FILE_PATH" != *"memory/index.md"* \
    && "$FILE_PATH" != *"memory/log.md"* \
    && "$FILE_PATH" != *"entity-registry.json"* \
    && "$FILE_PATH" != *".gitkeep"*
]]; then
    echo "REMINDER: memory/ was modified.
    Update memory/index.md and
    append to memory/log.md."
fi
exit 0    # ← never blocks
```

CUSTOM COMMANDS

Skills — Slash Commands

- Skills are markdown files in `.claude/skills/`. They define what Claude does when you type `/skill-name`.
- Loaded on-demand — not at startup. CLAUDE.md is always in context; skills are only loaded when invoked.
- CLAUDE.md defines *what and why*. Skills define *how* — the step-by-step procedure for a specific command.

```
---                                SKILL FILE FORMAT
name: research
description: Start a new research goal
user_invocable: true
---

# /research - Start a New Research Goal

## Gather Information
Ask the user for: goal, scope, criteria...

## Workflow
Step 1: Create GitHub issue
Step 2: Create branch
Step 3: Scaffold goal file
...
```

SKILL	PURPOSE
/research	New goal → GitHub issue → branch → goal file
/analyze	Ingest source → extract findings + entities
/synthesize	Consolidate findings → themes → output
/experiment	Design + run ML experiment → classify result
/distill	Extract decisions from settled findings
/evidence	Charts, tables, plots → <code>outputs/evidence/</code>
/verify	Code that checks claims → <code>outputs/verification/</code>
/examples	Worked examples → <code>outputs/examples/</code>
/critique	Adversarial review + gap analysis
/lint	Memory health: orphans, staleness, contradictions
/read-pdf	Render PDF → images → read figures + tables
/import	Bulk-import an existing project — copies to <code>sources/</code>

— PROJECT STRUCTURE

Repo Folder Layout

```

research-flow-template/
├── CLAUDE.md           # always loaded – the brain
├── .claude/            # hooks, skills, settings
├── .github/ISSUE_TEMPLATE/ # research-goal, hypothesis, finding
├── sources/            # ① IMMUTABLE raw material
├── memory/             # ② LLM-maintained wiki
│   ├── index.md        # catalog – updated every cycle
│   ├── log.md          # append-only operation log
│   ├── entity-registry.json # dedup index
│   ├── entities/       # people, orgs, concepts, tools
│   ├── findings/       # discrete insights + citations
│   ├── themes/         # cross-cutting patterns
│   ├── open-questions/ # gaps to investigate
│   └── decisions/      # distilled decisions
├── goals/              # active research goal files
├── outputs/            # deliverables
│   ├── evidence/
│   ├── verification/
│   ├── examples/
│   └── critiques/
├── experiments/        # ML training harness
│   ├── run.py          # primary entry point (YAML)
│   ├── configs/        # baselines/, sweeps/
│   └── results/         # run-log.jsonl + MLflow
└── docs/               # ③ Schema / conventions
  
```

TREE

THREE-LAYER MEMORY MODEL

1

Sources**sources/**

Immutable raw material. Write-once, never edited. Hook enforces this.

2

Working Memory**memory/**

LLM-maintained wiki. Updated every research cycle by Claude.

3

Schema**CLAUDE.md**

Conventions and rules. Loaded by Claude on every session.

— END-TO-END

A Full Research Session

- 1 You run `claude` in the repo root.
Claude loads `CLAUDE.md` and `.claude/settings.json` automatically.
- 2 You type `/research`.
Claude follows `skills/research.md` — creates a GitHub issue, branch, goal file, and log entry.
- 3 You type `/analyze https://...`.
Claude fetches the source, extracts findings + entities, writes to `memory/`.
- 4 `post-memory-update.sh` fires (PostToolUse → Edit).
Prints reminder. Claude updates `index.md` + `log.md` in the same turn.

- 5 If Claude edits `sources/` — `block-source-modification.sh` fires, cancels the edit with an error message.
- 6 If Claude runs `git commit` on main — `block-commit-protected-branch.sh` blocks it.
- 7 You type `/synthesize`.
Claude reads all findings, builds themes, resolves contradictions, writes an output report.
- 8 Parallel agents.
Spawn multiple agents on `hypothesis/` branches via `Agent({ isolation: "worktree" })`. Each works independently.

— EXTENDING THE SYSTEM

Adding Your Own Hooks

- 1 Write a shell script in `.claude/hooks/`.
Use `exit 0` to allow, `exit 1` to block.
- 2 Register it in `.claude/settings.json`
under the appropriate event and matcher.

```
# .claude/hooks/block-direct-output-write.sh
BASH - NEW HOOK
if [[ "$FILE_PATH" == *"outputs/"* ]]; then
  echo "BLOCKED: Use /evidence, /verify,
  /examples, or /critique to produce
  outputs - not direct Write."
  exit 1
fi
```

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write",
        "hooks": [{
          "type": "command",
          "command": "bash .claude/hooks/block-direct-output-write.sh Write \"\$TOOL_INPUT_FILE_PATH\"
        }]
      }
    ]
  }
}
```

Hooks run as the current user with access to the full shell environment. Keep them fast — they run on every matching tool call.

— EXTENDING THE SYSTEM

Adding Your Own Skills

- 1 Create `.claude/skills/my-skill.md` with YAML frontmatter and a markdown body describing the workflow.
- 2 Invoke with `/my-skill` in the Claude Code prompt. No registration needed beyond the file.
- 3 Skills can reference CLAUDE.md conventions freely — Claude has CLAUDE.md in context when the skill runs.

Skills vs CLAUDE.md: If Claude needs to know something every session → CLAUDE.md. If it's a procedure only needed when invoked → skill.

MARKDOWN — SKILL FILE

```
---  
name: my-skill  
description: One-line description for the picker  
user_invocable: true  
---
```

```
# /my-skill — What This Does
```

```
## Gather Information  
Ask the user for: ...
```

```
## Workflow
```

```
### Step 1  
Do X, Y, Z.
```

```
### Step 2  
Write to memory/findings/ using the  
template in docs/memory-page-template.md.
```

```
### Step 3  
Update memory/index.md and append  
to memory/log.md.
```

NEW SKILL

/import — Bring In an Existing Project

/analyze ingests one source at a time. **/import** bulk-ingests a whole project in one pass — maps the structure, classifies artifacts, builds memory.

- 1 Accepts local path, git URL, or glob — `../my-project` , `https://github.com/org/repo` , `./docs/**/*.md`
- 2 Classifies artifacts into tiers — Primary docs, Code, Data, Config, Skip. Copies Primary + Code to `sources/<slug>/` .
- 3 Extracts entities and findings from READMEs, notebooks, design docs, comments — with verification gates + FUNGI counter-arguments.
- 4 Mines open questions from TODO/FIXME comments, "Future Work" sections, skipped tests, missing docs.

ARTIFACT TIERS		
TIER	WHAT	ACTION
Primary	READMEs, notes, notebooks, papers	Copy + deep analysis
Code	Source files (.py, .ts, ...)	Copy key files + summarize
Config	pyproject.toml, YAML, package.json	Copy + extract deps
Data	Datasets, CSVs, large files	Pointer file only
Skip	.git/, node_modules/, __pycache__/	Ignore

Local project

Git repo (cloned to /tmp/ first)

Just the docs

After import, continue with:

USAGE

```
/import ../my-existing-project

/import https://github.com/org/repo

/import ./my-project/docs/**/*.md

/import # deep-dive a specific file
```

— /IMPORT — STEP BY STEP

/import — How It Works

- 1 Resolve & inspect the source. List the directory or clone the git repo to `/tmp/import-<slug>`. Build a map of file types and sizes.
- 2 Classify artifacts into tiers. Primary docs → deep analysis. Code → architecture summary. Data → pointer stub. `node_modules/`, `.git/` → skipped.
- 3 Copy to `sources/<slug>/` with a `MANIFEST.md` listing every file imported and every file skipped (with reason). Sources are immutable after this step.
- 4 Extract entities. Check `entity-registry.json` before creating. Tools, libraries, models, datasets, authors — each gets a page in `memory/entities/`.
- 5 Extract findings from READMEs, notebooks, design notes, and code comments. Every finding gets a verification gate (`source` / `analysis` / `unverified`) and a FUNGI counter-argument.
- 6 Mine open questions from TODO/FIXME comments, "Future Work" sections, skipped tests, and missing documentation. Each goes to `memory/open-questions/`.
- 7 Update memory index & log. All new pages catalogued in `memory/index.md`. The full operation appended to `memory/log.md`.
- 8 Optionally create a research goal. If you want a structured research branch around the imported project, /import can hand off to `/research` automatically.

After /import, use **/analyze** to deep-dive any specific file, or **/experiment** if the project has ML training code.

Further Reading

OVERVIEW

What Claude Code is, how it works, and the core concepts (CLAUDE.md, hooks, skills, MCP).

docs.anthropic.com/en/docs/claude-code/overview

HOOKS

Full hook event types (PreToolUse, PostToolUse, SessionStart, UserPromptSubmit, ...), exit code semantics, environment variables.

docs.anthropic.com/en/docs/claude-code/hooks

SLASH COMMANDS (SKILLS)

Creating custom slash commands, `.claude/commands/` structure, argument passing with `$ARGUMENTS`.

docs.anthropic.com/en/docs/claude-code/slash-commands

MEMORY & CLAUDE.MD

CLAUDE.md scopes (managed policy, project, user, local), memory types, and how context is loaded at session start.

docs.anthropic.com/en/docs/claude-code/memory

SETTINGS

Full settings.json reference: permission modes, allowed tools, hook wiring, MCP servers, and environment config.

docs.anthropic.com/en/docs/claude-code/settings

THIS TEMPLATE

Scaffold a new research project with the `create-research-flow` CLI — creates the repo, GitHub Project board, and opens Claude Code.

github.com/dangquan1402/create-research-flow

BEFORE YOU BEGIN

Prerequisites

REQUIRED

Claude Code

CLI

The AI agent that drives everything. Requires a **Claude.ai subscription** (Pro or Max) — sign in with `claude` after install.

```
npm install -g @anthropic-ai/claude-code && claude
```

Node.js ≥ 18

for npx

Needed to run the `npx create-research-flow` scaffold.

```
node --version # should print v18+
```

Python ≥ 3.12 + uv

experiment harness

For `uv sync` and the PDF tooling. Add your own ML framework (torch, jax, etc.) as needed.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

GitHub CLI (gh)

issue tracking

Creates the GitHub repo, Project board, labels, and links branches to issues.

```
brew install gh && gh auth login
```

OPTIONAL

Google Chrome

PDF export

Only needed to re-export this deck to PDF via headless Chrome. Not required for research work.

AUTHENTICATION

Claude Code — log in with your claude.ai account `claude` # opens browser auth on first run # GitHub — for Project board & issue tracking `gh auth login` # sets GITHUB_TOKEN automatically

Check everything: `node -v && gh --version && uv --version && claude --version`